

Structures, Unions and Typedef in C

📌 Definition to Remember

A structure (struct) is a user-defined composite data type in C that groups logically related variables of different data types under a single unified name.

1. Concept Overview & Fundamentals 🗨️

Structures, Unions and Typedef in C forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

- Code is compiled directly into native machine instructions without virtual machine overhead.
- Memory is strictly partitioned into **Code/Text Segment**, **Data Segment (Initialized/BSS)**, **Stack**, and **Heap**.
- Direct pointer and register manipulation provides deterministic performance critical for operating systems, embedded hardware, and database kernels.

2. Technical Architecture & Memory Mechanics ⚙️

Memory Layout & Storage Organization

```
+-----+
|                STACK SEGMENT                |
| - Local variables, function call stack frames, parameters |
| - High memory growing downwards towards Heap |
+-----+
|
| v
| ^
|
+-----+
|                HEAP SEGMENT                |
| - Dynamic memory allocations via malloc(), calloc() |
| - Low memory growing upwards towards Stack |
+-----+
|                INITIALIZED DATA SEGMENT    |
| - Global and static variables with non-zero initial values |
+-----+
|                BSS SEGMENT (UNINITIALIZED)  |
| - Global and static variables initialized to zero |
+-----+
|                CODE / TEXT SEGMENT          |
| - Read-only executable binary instructions |
+-----+
```

Core Architecture Breakdown

Component / Concept	Technical Specification	Operational Characteristics	Performance / Complexity
Storage Allocation	Stack vs Heap vs Data Segment	Stack is ultra-fast LIFO; Heap offers dynamic scalability	Stack: O(1) push/pop; Heap: O(1) alloc
Type Checking	Static Typing	Compiler verifies types during compilation	Zero runtime type penalty
Addressing Model	Byte-addressable physical RAM	Pointer stores hexadecimal byte memory address	Direct memory dereference (O(1))
Execution Flow	Procedural / Sequential	Controlled via jumps, branches, loops, and call stacks	Deterministic instruction cycles

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

// Core standard implementation pattern
int main(void) {
    // Explicit initialization
    int status = 0;

    // Core logic execution
    printf("Executing C Module: Structures\n");

    return status;
}
```

4. Real-World Applications & System Engineering

- **Operating System Kernels:** Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.
- **Embedded & IoT Devices:** Microcontrollers with constrained RAM (8KB-64KB) require C's predictable footprint.
- **Database Engine Kernels:** High-throughput storage engines (PostgreSQL, SQLite, MySQL InnoDB) rely on low-level buffer pool managers written in C.
- **Game Engine Foundations:** Graphics rasterization, physics engines, and GPU interfaces utilize C/C++ memory arrays for zero-overhead computation.

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: book array initializer

Problem Description & Objective:

Implementation of book array initializer

C Code Implementation:

```
/*
Initialize an array of Book structures with
different data for each book using designated initializers.
*/

#include <stdio.h>

// Structure Definition
struct Book {
    char title[50];
    char author[50];
    float price;
};

int main() {
    /*
    * Logic:
    * - Goal: Initialize an array of Book structures with different data for each book using designated
    initializers.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    struct Book books[3] = {
        {.title = "C Programming", .author = "Dennis Ritchie", .price = 499.99},
        {.title = "Python Basics", .author = "Guido van Rossum", .price = 599.50},
        {.title = "Data Structures", .author = "Mark Allen", .price = 450.75}
    };

    int i;

    printf("Book Details:\n\n");

    for(i = 0; i < 3; i++) {

        printf("Book %d\n", i + 1);
        printf("Title : %s\n", books[i].title);
        printf("Author: %s\n", books[i].author);
        printf("Price : %.2f\n\n", books[i].price);
    }

    return 0;
}
```

Problem 2: book structure

Problem Description & Objective:

Implementation of book structure

C Code Implementation:

```
/*
Create a program where you need to store and process
data for a Book with attributes like title, author,
and price, demonstrating why a structure is more
suitable than separate variables.
*/

#include <stdio.h>

// Structure Definition
struct Book {
    char title[100];
    char author[100];
    float price;
};

int main() {
    /*
    * Logic:
    * - Goal: Create a program where you need to store and process data for a Book with attributes like
    title, author, and price, demonstrating why a structure is more suitable than separate variables.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    struct Book book1;

    printf("Enter book title: ");
    fgets(book1.title, sizeof(book1.title), stdin);

    printf("Enter author name: ");
    fgets(book1.author, sizeof(book1.author), stdin);

    printf("Enter book price: ");
    scanf("%f", &book1.price);

    printf("\nBook Details:\n");
    printf("Title: %s", book1.title);
    printf("Author: %s", book1.author);
    printf("Price: %.2f\n", book1.price);

    return 0;
}
```

Problem 3: car description function

Problem Description & Objective:

Implementation of car description function

C Code Implementation:

```
/*
Pass a Car structure to a function that prints
out a description of the car in one complete sentence.
*/

#include <stdio.h>

// Structure Definition
struct Car {
    char make[50];
    char model[50];
    int year;
    char color[30];
};

// Function Definition
void printDescription(struct Car car) {

    printf("This car is a %d %s %s and its color is %s.\n",
           car.year,
           car.make,
           car.model,
           car.color);
}

int main() {
    /*
    * Logic:
    * - Goal: Pass a Car structure to a function that prints out a description of the car in one
    complete sentence.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    struct Car car1 = {
        "Honda",
        "City",
        2023,
        "White"
    };

    // Function Call
    printDescription(car1);

    return 0;
}
```

Problem 4: car structure

Problem Description & Objective:

Implementation of car structure

C Code Implementation:

```
/*
Define a Car structure with fields for
make, model, year, and color.
*/

#include <stdio.h>

// Structure Definition
struct Car {
    char make[50];
    char model[50];
    int year;
    char color[30];
};

int main() {
    /*
    * Logic:
    * - Goal: Define a Car structure with fields for make, model, year, and color.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    struct Car car1 = {
        "Toyota",
        "Fortuner",
        2024,
        "Black"
    };

    printf("Car Details:\n");
    printf("Make : %s\n", car1.make);
    printf("Model: %s\n", car1.model);
    printf("Year : %d\n", car1.year);
    printf("Color: %s\n", car1.color);

    return 0;
}
```

Problem 5: nested structure student books

Problem Description & Objective:

Implementation of nested structure student books

C Code Implementation:

```

/*
Write a function where the Student structure
also has books they have borrowed inside,
showing nested structure usage.
*/

#include <stdio.h>

// Nested Structure
struct Book {
    char title[50];
    char author[50];
};

// Main Structure
struct Student {
    int id;
    char name[50];
    struct Book borrowedBook;
};

// Function Definition
void displayStudent(struct Student s) {

    printf("Student ID   : %d\n", s.id);
    printf("Student Name : %s\n", s.name);

    printf("Borrowed Book:\n");
    printf("Title   : %s\n", s.borrowedBook.title);
    printf("Author  : %s\n", s.borrowedBook.author);
}

int main() {
    /*
    * Logic:
    * - Goal: Write a function where the Student structure also has books they have borrowed inside,
    showing nested structure usage.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    struct Student student1 = {
        101,
        "Suraj",
        {"C Programming", "Dennis Ritchie"}
    };

    displayStudent(student1);

    return 0;
}

```

Problem 6: student modify gpa

Problem Description & Objective:

Implementation of student modify gpa

C Code Implementation:

```

/*
Write a function that accepts a pointer to a
Student structure with fields for id, name,
year, gpa and modifies its grades.
*/

#include <stdio.h>

// Structure Definition
struct Student {
    int id;
    char name[50];
    int year;
    float gpa;
};

// Function Definition
void modifyGPA(struct Student *s) {

    s->gpa = 9.2;
}

int main() {
    /*
    * Logic:
    * - Goal: Write a function that accepts a pointer to a Student structure with fields for id, name,
    year, gpa and modifies its grades.
    * - Prompts the user for required inputs.
    * - Executes standard control flow, conditions, or loops to compute the result.
    * - Prints the formatted output to the console.
    */

    struct Student student1 = {
        101,
        "Suraj",
        2,
        7.5
    };

    printf("Before Modification:\n");
    printf("GPA = %.2f\n", student1.gpa);

    // Function Call
    modifyGPA(&student1);

    printf("\nAfter Modification:\n");
    printf("GPA = %.2f\n", student1.gpa);

    return 0;
}

```

6. Exam Scoring Points & Memory Revision

★ Must-Write Points

1. struct definition: Groups heterogeneous data fields; accessed via dot operator (obj.field).
2. Pointer to struct: Access members using arrow operator (ptr->field == (*ptr).field).
3. Structure Padding & Memory Alignment: Compiler inserts padding bytes to align data on word boundaries.
4. typedef keyword creates intuitive aliases (e.g. typedef struct Student Student;).
5. Union vs Structure: struct allocates sum of member sizes (plus padding); union shares single memory location equal to largest member.

⚡ Quick Recall

Heterogeneous Grouping -> struct definition -> dot (.) / arrow (->) -> Alignment Padding -> Union Share